UNIT V

LECTURE 1

DISTRIBUTED AGREEMENT

Introduction:

This is used to elect a co ordinator process or deciding to commit a transaction in distributed system. It uses a majority voting mechanism which can tolerate a faulty process.

System Fault means a malfunction or error in UNISS such that the System is unable to perform the functions it is required to perform for the time being. Failure handling settings allow users to decide whether Katalon Studio will continue running or not in case of errors occurs during execution. System failures may result from a hard drive with bad sectors, causing the operating system to not be able to read data from the hard drive. A failing motherboard can cause a system failure because the computer is not able to process requests or operate in general. A system failure is a problem either with hardware (other than disk) or with operating system software that causes your system to end abnormally.

BYZANTINE FAULTS

- **Byzantine Fault** Tolerance(BFT) is the feature of a **distributed** network to reach consensus (agreement on the same value) even when some of the nodes in the network fail to respond or respond with incorrect information.
- DS are subject to a variety of faults and attacks.
- In this we consider general faulty node which may exhibit arbitrary behavior.
- The basis of **Byzantine Fault Tolerance** is **achieved** when an incoming message is repeated by all the nodes. If a node is repeating the incoming message, that means it is

not **faulty** or malicious. If all the recipients repeat the incoming message, the network rules out the issue of **Byzantine** nodes.

Failure and Recovery

Failure of a site/node in a distributed system causes inconsistencies in the state of the system.Recovery: bringing back the failed node in step with other nodes in the system.Failures:

Process failure: Deadlocks, protection violation, erroneous user input, etc.

System failure: Failure of processor/system. System failure can have full/partial amnesia. It can be a pause failure (system restarts at the same state it was in before the crash) or a complete halt.

Secondary storage failure: data inaccessible.

Communication failure: network inaccessible.

UNIT V

LECTURE II

BYZANTINE AGREEMENT

One of the fundamental problems in fault tolerant distributed computing is the Byzantine agreement problem. Byzantine agreement requires a set of parties in a distributed environment to agree on a value even if some of the parties are corrupted. A number of solutions to the Byzantine agreement protocol exist.

Byzantine fault (also Byzantine generals problem, interactive consistency, source congruency, error avalanche, Byzantine agreement problem, and Byzantine failure) is a condition of a computer system, particularly distributed computing systems, where components may fail.

he Byzantine Agreement protocol is **a protocol in distributed computing**. It takes its name from a problem formulated by Lamport, Shostak and Pease in 1982, which itself is a reference to a historical problem.

To achieve reliability, the fault-tolerance scheme of the distributed computing system must be revised. This kind of problem is known as a Byzantine agreement (BA) problem. **It requires all fault-free processors to agree on a common value, even if some components are corrupt**.

A Byzantine fault is defined as **an arbitrary fault that occurs during the execution of an algorithm by a distributed system**. When a Byzantine failure occurs, the system may respond in any arbitrary way unless it is designed to have Byzantine fault tolerance.

Agreement Protocols

- In distributed systems, sites are often required to reach mutual agreement
  - In distributed database systems, data managers must agree on whether to commit or to abort a transaction
  - Reaching an agreement requires the sites have knowledge about values at other sites
- Agreement when the system is free from failures
- Agreement when the system is prone to failure

- The Agreement or the Consensus Problem:
- Assume P = {p 1,…,p M } is the set of all the processors in the system Some processors in P are faulty Let F be the set of all faulty processors in P Every processor p in P has a value p.Val
- The requirement: devise a distributed algorithm that lets each processor p computes a value p.A such that when the execution of this distributed algorithm terminates, the following two conditions hold:
- 1. (agreement value) For every pair of processors p and q, q.A = p.A
- 2. The agreement value is a function of the initial values {p.Val} of non-faulty processors.
- Adversaries:- it deals with the control of environment in which protocol or agreement executes.

- Byzantine Agreement: Assumptions A failed processor can send arbitrary messages A non-failed processor always responds to a message within T seconds When a processor receives a message, it can reliably determine the sender of that message.

- Send(p,m) receive(m,p)

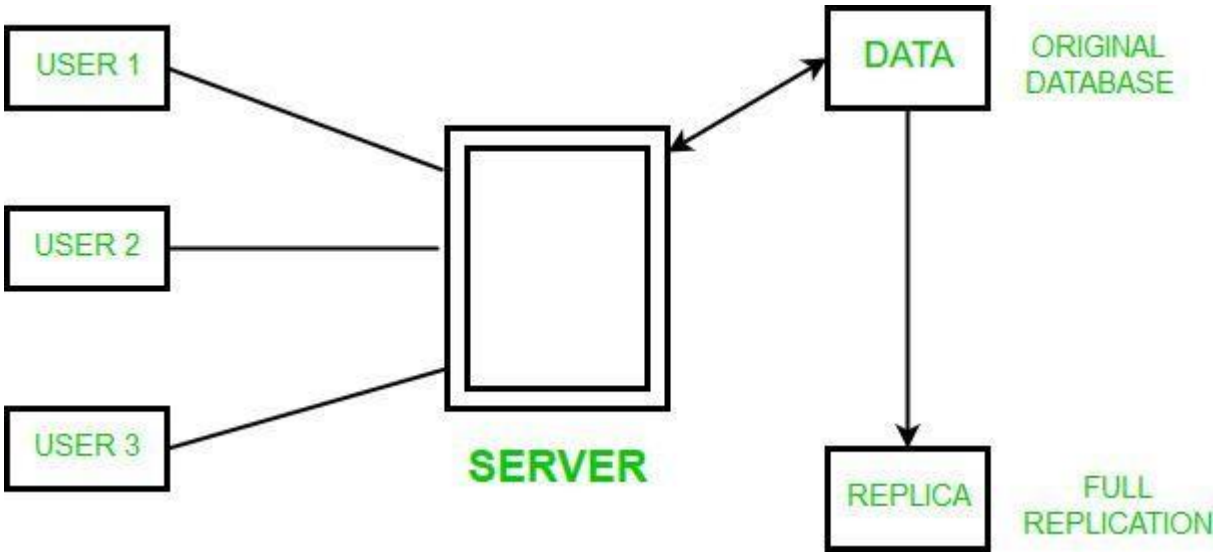
UNIT V

LECTURE III

Replicated Data Management Concept and Issues:

Replication is the key factor in improving the availability of data in distributed systems. Replicated data is stored at multiple sites so that it can be accessed by the user even when some of the copies are not available due to site failures.

Data Replication is **the process of storing data in more than one site or node**. It is useful in improving the availability of data. It is simply copying data from a database from one server to another server so that all the users can share the same data without any inconsistency.



Replication of data in a distributed system is a way to enhance the performance of applications that access the data. A system where data is replicated can provide better fault tolerance capabilities as well as improved response time. However, such improvement is achieved at the expense of having to manage replication by implementing replica control protocols. Such protocols are required to

insure that data consistency is maintained in the face of system failures. In this article we describe the issues involved in maintaining the consistency of a replicated database system. We next describe three basic techniques for managing replicated data and discuss the relative merits of each technique. This is followed by a survey of extensions to the basic approaches. A discussion of future directions in research on data replication concludes our presentation.

- As discussed earlier, **replication** is a technique used in **distributed databases** to store multiple copies of a data table at different sites. The problem with having multiple copies in multiple sites is the overhead of maintaining data consistency, particularly during update operations.
- Synchronous **system** updates all the replicas before the transaction commits. Updates to all replicas are treated in the same way as any other **data** item.
- Synchronous **systems** produce globally serializable schedules.
- In asynchronous **systems**, only a subset of the replicas is updated.
- Types of replication:
- Snapshot **replication** sends **the** entire data set to **the** subscriber.
- Transactional **replication** only sends modifications to **the** data.
- Merge **replication** items are modified at both **the** publisher and subscribers.
- Heterogeneous **replication** allows access to **other** database products.

UNIT V

LECTURE IV

Atomic Multicast

Atomic multicast is a communication primitive that delivers messages to multiple groups of processes according to some total order, with each group receiving the projection of the total order onto messages addressed to it.

Atomic multicast is a group communication abstraction that groups processes, providing reliabil- ity and ordering guarantees, and can be explored to provide partially replicated applications a scalable and consistent alternative.

reliable multicast is any computer networking protocol that provides a reliable sequence of packets to multiple recipients simultaneously, making it suitable for applications such as multi-receiver file transfer.

Why Consensus is needed?

It ensures that you will never converge to an incorrect value or correct individuals in a network will never converge to an incorrect value. It states that every correct value must be accepted eventually which means something good will eventually happen.

Multicast protocols such as UDP/IP and RTP/IP are used to broadcast data to any number of receiving devices. Unlike unicast TCP data, if a packet is not received, there is no mechanism or attempt to verify that and resend the packet. Multicast is widely used for video and audio data.

- **Atomic multicast** is a communication building block that allows messages to be propagated to groups of processes with reliability and order guarantees.

- **Atomic multicast** is a communication primitive that delivers messages to multiple groups of processes according to some total order, with each group receiving the projection of the total order onto messages addressed to it.
- A genuine **atomic multicast** is the foundation of scalable **systems**, since it does not depend on a fixed group of processes and does not involve all processes.
- Different copies can be kept in sync using an **atomic** broadcast protocol, which delivers application messages to replicas in some total order and thereby ensures that they evolve in the same way .....In this setting, **replica** consistency can be maintained using **atomic multicast**.

Multicast Routing Protocols

- Forwarding decisions: different from unicast protocols
  - multicast protocols use the source address instead of the destination address to make their forwarding decisions
- Two main protocols
  - DVMRP (*Distance Vector Multicast Routing Protocol*)
    - Distance vector (RIP-like) algorithm
    - Static configuration based on tunnels between DVMRP routers (often Sun workstations)
    - Not easily scalable (e.g., max hops < 32)
  - PIM (*Protocol Independent Multicast*)
    - Typically installed on routers
    - Independent from the unicast routing protocols used by the router
    - Scalable (*Dense-mode* & *Sparse-mode*)

UNIT V

LECTURE V

CASE STUDY: CORBA

INTRODUCTION:

The abbreviation of CORBA is

- **CORBA** (COMMON OBJECT REQUEST BROKER ARCHITECTURE)
- CORBA is a middleware design that allows application programs to communicate with one another irrespective of their programming languages, their hardware and software platforms, the networks they communicate over and their implementers.
- Applications are built from CORBA objects, which implement interfaces defined in CORBA's interface definition language, IDL. Clients access the methods in the IDL interfaces of CORBA objects by means of RMI.
- The middleware component that supports RMI is called the Object Request Broker or ORB.
- The specification of CORBA has been sponsored by members of the Object Management Group (OMG).
- Many different ORBs have been implemented from the specification, supporting a variety of programming languages including Java and C++.
- CORBA services provide generic facilities that may be of use in a wide variety of applications. They include the Naming Service, the Event and Notification Services, the Security Service, the Transaction and Concurrency Services and the Trading Service.

Role of the ORB:

- An important motivation was to allow distributed objects to be implemented in any programming language and to be able to communicate with one another.
- They therefore designed an interface language that was independent of any specific implementation language. They introduced a metaphor, the object request broker (or ORB), whose role is to help a client to invoke a method on an object.

- This role involves locating the object, activating the object if necessary and then communicating the client's request to the object, which carries it out and replies.
- The main components of CORBA's language-independent RMI framework are the following:
  • An interface definition language known as IDL.
- The CORBA architecture also allows for CORBA services – a set of generic services that can be useful for distributed applications.

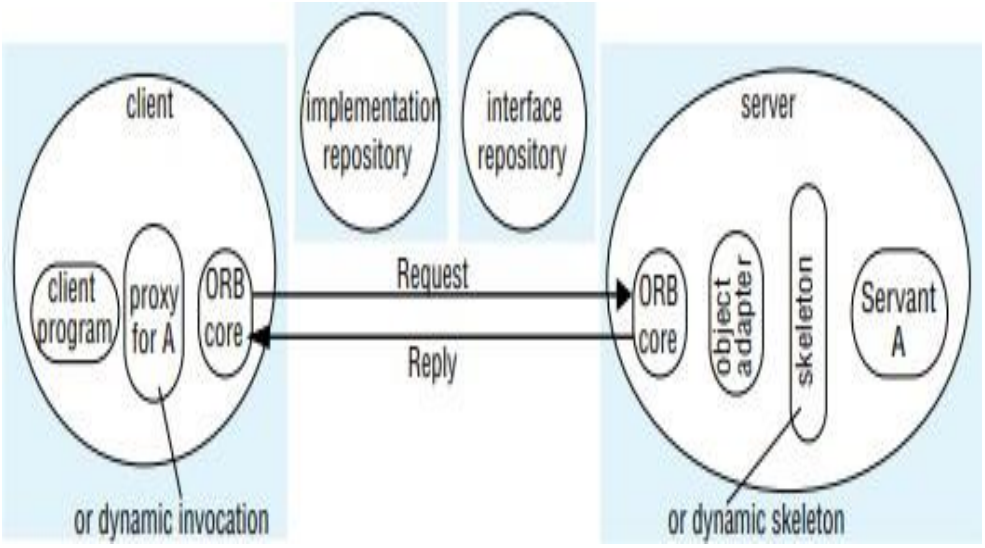
UNIT V

LECTURE VI

CORBA RMI

- Programming in a multi-language RMI system such as CORBA RMI requires more of the programmer than programming in a single-language RMI system such as Java RMI. The following new concepts need to be learned:
- the object model offered by CORBA;
- the interface definition language and its mapping onto the implementation language.
- In CORBA object model, clients are not necessarily objects – a client can be any program that sends request messages to remote objects and receives replies.
- The term CORBA object is used to refer to remote objects. Thus, a CORBA object implements an IDL interface, has a remote object reference and is able to respond to invocations of methods in its IDL interface.

CORBA COMPONENT:



CORBA:

CORBA is a middleware design that allows application programs to communicate with one another irrespective of their programming languages, their hardware and software platforms, the networks they communicate over and their implementers.

- The CORBA architecture allows CORBA objects to be activated on demand. This is achieved by a component called the implementation repository, which keeps a database of implementations indexed by their object adapter names.
- When a client invokes a CORBA object, it can be activated if necessary in order to carry out the invocation. An interface repository is a database of IDL interface definitions indexed by repository IDs.
- Since the IOR of a CORBA object contains the repository ID of its interface, the appropriate interface repository can be used to get the information about the methods in its interface which is required for dynamic method invocations.
- CORBA services provide functionality above RMI, which may be required by distributed applications, allowing them to use additional services such as naming and directory services, event notifications, transactions or security as required.

CORBA services:

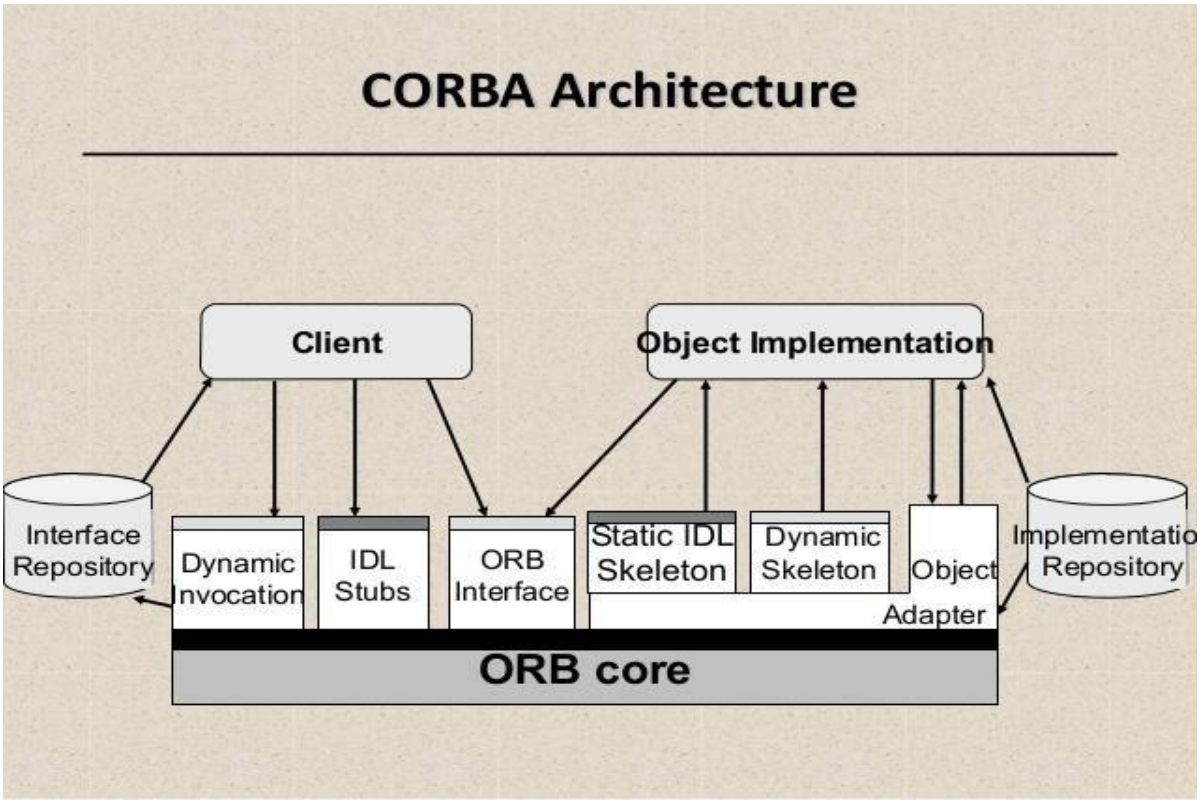
- **Naming Service**:-it allows names to be bound to the remote object reference of CORBA object within naming contacts.
- **Event Service**:-it defines the interest of object to communicate to the notification subscriber.
- **Notification Service**:- it defines the data structure(event supplier)are provided the discovered event, consume and interested in it.
- **Security Service**:-(A) Authentication:- By authentication we have to provide the security service.
- (B) Access Control:-it can be applied with in the CORBA object, when they receive RMI's .

UNIT V

LECTURE VII

Applications of CORBA

- Used primarily as a remote method invocation of a distributed client – server system.
- Can communicate between clients and servers on different operating systems and implemented by different programming languages (Java cannot do this).
- Has many standards and services useful in implementing distributed appliations.
- Process can be both server and client to another server
- Ideal for a heteregeneous distributed system like the Internet.



DISCRIPTIONS OF CORBA ARCHITECTURE

- A CORBA object implements the operations in an IDL interface.
- All that clients need to know to access a CORBA object is the operations available in its interface.
- The client program accesses CORBA objects via proxies or stubs, which are generated automatically from their IDL interfaces in the language of the client.
- Server skeletons for CORBA objects are generated automatically from their IDL interfaces in the language of the client.
- The object adapter is an important component of CORBA servers.
- Its roles include activating and deactivating servants, creating remote object references and forwarding request messages to the appropriate servants.
- The main component of CORBA is the Object Request Broker or ORB, which allows clients written in one language to invoke operations in remote objects (called CORBA objects) written in another language. CORBA addresses other aspects of heterogeneity as follows:
- The CORBA General Inter-ORB protocol (GIOP) includes an external data representation called CDR, which makes it possible for clients and servers to communicate irrespective of their hardware. It also specifies a standard form for remote object references.
- GIOP also includes a specification for the operations of a request-reply protocol that can be used irrespective of the underlying operating system.
- The Internet Inter-ORB Protocol (IIOP) implements the request-reply protocol over TCP. IIOP remote object references include the domain name and port number of a server.

UNIT V

LECTURE VIII

Features

- CORBA consists of a language independent RMI
- Consists of a set of generic services useful for distributed applications.
- The CORBA RMI acts as a "universal translator" that permits client processes to invoke a method or process that may reside on a different operating system or hardware, or implemented via a different programming language.

Introduction to CORBA

- The Object Management Group (OMG) was formed in 1989. Its aims were:
  - to make better use of distributed systems
  - to use object-oriented programming
  - to allow objects in different programming languages to communicate with one another
- The object request broker (ORB) enables clients to invoke methods in a remote object
- CORBA is a specification of an architecture supporting this.
  - CORBA 1 in 1990 and CORBA 2 in 1996.

CORBA RMI Features

- The CORBA RMI consists of the following main components:
  - An interface definition language (IDL)
  - An architecture (discussed in **Structure**)
  - The General Inter-ORB Protocol (GIOP)
  - The Internet Inter-ORB Protocol (IIOP)

5.1 CORBA RMI

❑ The Object Management Group (OMG)I:

- **OMG introduced the Object Request Broker (ORB).**
  - **ORB is a distributed object model component with a role involving:**
  - **Locating a object.**
  - **Activating the object.**
  - **Passing the client request to the object which executes it and send a reply.**
- **The aim was to enable:**
  - **Distributed objects to be implemented in any programming language.**
  - **And for such objects to communicate with each other.**

CORBA RMI

- CORBA RMI is a multi-language RMI system.
- The programmer needs to learn the following new concepts:
  - the object model offered by CORBA;
  - the interface definition language and its mapping onto the implementation language. (e.g. a *struct* in IDL is mapped onto what in Java?)
- CORBA's object model
  - similar to the remote object model in Chapter 5 (what are the main features?)
  - clients are not necessarily objects (why not?)– a client can be any program that sends request messages to remote objects and receives replies.
- The term *CORBA object* is used to refer to remote objects.
  - *a CORBA object* implements an IDL interface, has a remote object reference and its methods can be invoked remotely.
- A CORBA object can be implemented by a language without classes.
  - the class concept does not exist in CORBA.
  - therefore classes cannot be defined in CORBA IDL, which means that instances of classes cannot be passed as arguments.

